P2809R1
Trivial infinite loops are not Undefined Behavior

Published Proposal, 2023-06-18

This version:
http://wg21.link/P2809r1

Author:
JF Bastien (Woven by Toyota)

Audience:
SG1, SG22, EWG, LEWG

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Source:
github.com/jfbastien/papers/blob/master/source/P2809r1.bs

Table of Contents

§ 1. Introduction

C and C++ diverge in their definition of forward progress guarantees, and have done so since C++11 and C11 added a memory model and acknowledged that threads exist. Both committees have been working together to reduce needless differences. [P2735r0] and [N2644] perform such harmonization.

C does not make iteration statements whose controlling expression is a constant expression Undefined Behavior, whereas C++ does. C++ implementations therefore assume that even "trivial" infinite loops must terminate.

This is unfortunate because using an infinite loop is a common idiom in low-level programming, particularly when a bare-metal system or kernel must halt progress. It would be easy for programmers to satisfy the C++ requirements on infinite loops, and it would be easy for compilers to diagnose "invalid" infinite loops. Neither programmers nor implementations

have done so. Instead, programmers decry C++'s obtuseness, and implementations upset developers by optimizing away their infinite loops (marking them as unreachable, leading to surrounding code being optimized away).

For example, this code prints `Hello, World!` in recent versions of clang (see example):

```
#include <iostream>

int main() {
  while (true)
    ;
}

void unreachable() {
  std::cout << "Hello world!" << std::endl;
}
```

It is easy for the C++ Standards Committee to instead align with C, and allow "trivial" infinite loops as C does. However, care must be taken when doing so. C++ has good reasons to specify that infinite loops are undefined behavior: it is desirable to express forward progress guarantees in a programming language because it unlocks a design space in concurrency and parallelism software and hardware design. An 80 minute deep-dive in this topic is available in [ForwardProgress]. A definition of "trivial" which continues supporting this design space is important to reach.

Other programming languages, compilers, and hardware explicitly or de-facto rely on the C++ specification of its abstract machine, including the behavior infinite loops. Some by borrowing from C++'s specification, others by simply using compilers that implement C++ as their optimizer (see [LLVMProgressDiscussion], and a Rust bug of this shape), and yet others by being designed around C++ programs. It is therefore likely that a broader specification for an abstract machine, outside of C++, is needed to truly resolve problems as those tackled in this paper.

§ 2. Revision History

[P2809r0] proposed matching the C semantics. It was discussed at the 2023 Varna meeting and received feedback from SG1 against matching C semantics, as explained below.

§ 3. Standards

Since C++11, the Standard's forward progress guarantees have been defined in **[intro.progress]** as follows:

> The implementation may assume that any thread will eventually do one of the following:
>
> - terminate,
> - make a call to a library I/O function,
> - perform an access through a `volatile` glvalue, or
> - perform a synchronization operation or an atomic operation.
>
> [*Note:* This is intended to allow compiler transformations such as removal of empty loops, even when termination cannot be proven. —*end note*]

This forward progress guarantee applies to the following iteration statements:

> `while` `(` *condition* `)` *statement*
> `do` *statement* `while` `(` *expression* `)` `;`
> `for` `(` *init-statement* $condition_{opt}$ `;` $expression_{opt}$ `)` *statement*
> `for` `(` $init\text{-}statement_{opt}$ *for-range-declaration* `:` *for-range-initializer* `)` *statement*

Since C11, the Standard's forward progress guarantees have been defined in **Iteration statements** (section 6.8.5 of C17) as follows:

> An iteration statement may be assumed by the implementation to terminate if its controlling expression is not a constant expression[†], and none of the following operations are performed in its body, controlling expression or (in the case of a `for` statement) its *expression-3*[‡]:
>
> - input/output operations
> - accessing a `volatile` object
> - synchronization or atomic operations.
>
> [†] An omitted controlling expression is replaced by a nonzero constant, which is a constant expression.
>
> [‡] This is intended to allow compiler transformations such as removal of empty loops even when termination cannot be proven.

This forward progress guarantee applies to the following iteration statements:

> `while (` *expression* `)` *statement*
> `do` *statement* `while (` *expression* `) ;`
> `for (` $expression_{opt}$ `;` $expression_{opt}$ `;` $expression_{opt}$ `)` *statement*
> `for (` *declaration* $expression_{opt}$ `;` $expression_{opt}$ `)` *statement*

By omission, iteration statements are undefined if the controlling expression does not meet the criteria above.

§ 4. History

C99 6.8.5p4 states:

> An iteration statement causes a statement called the loop body to be executed repeatedly until the controlling expression compares equal to `0`.

C99 therefore disallows compilers from assuming that loops will eventually terminate.

C++11 added a full memory model to C++, allowing C++ to speak of threads and inter-thread communications in a useful manner. This forward progress guarantee does away with the C99 guarantee above. A key to concurrency and parallelism is understanding when forward progress is guaranteed, because without forward progress some concurrent and parallel algorithms do not work. For example, a pair of producer / consumer threads will never consume data if the producer does not have forward progress.

This memory model work was imported into the C11 standard through [N1349]. There were discussions on the topic of forward progress:

- [N1509] proposed eliminating the paragraph on forward progress from C1X.
- [N1528] responded to N1509, explaining the C1X wording.

These two papers were discussed, and WG14 took the following decision:

> **ACTION** Douglas to work with Larry to come up with the proposed words are:
>
> Change 6.8.5p6 as follows:
>
> > An iteration statement whose controlling expression is not a constant expression (or omitted), that performs no input/output operations, does not access `volatile` objects, and performs no synchronization or atomic operations in its body, controlling expression, or (in the case of a `for` statement) its *expressionX3*, may be assumed by the implementation to terminate.
>
> **Decision:** Adopt N1509, as modified above, to the C1X WP. result: 14 favor, 0 neutral, 1 against

§ 5. Rationale

The forward progress guarantees were added to C++11 and C11 along with a memory model. This addition of forward progress guarantees reflected existing practice in compilers, and did not simply serve concurrency and parallelism goals. Indeed, compilers had been assuming that loops can terminate to enable optimizations that transform loops and reorder store instructions and other side-effects. Proving that loops terminate is generally equivalent to solving the halting problem. The compilers were therefore disregarding the C99 wording which said that loops terminate when their controlling expression becomes `0`, and were therefore non-conformant.

The standards added the forward progress guarantees to change an optimization problem from "solve the halting problem" to "there will be observable side effects in the forms of termination, I/O, `volatile`, and/or atomic synchronization, any other operation can be reordered". The former is generally impossible to solve, whereas the latter is eminently tractable.

Forward progress guarantees therefore helped program concurrency and parallelism, and standardized existing practice which unlocked performance in compiler optimizations.

However, developers do sometimes rely on infinite loops, for valid reasons. When they do so, the loops have controlling expressions which are constant expressions. Such controlling expressions are known at compile-time by definition, and are therefore easy for a compiler to detect, and mark a loop as not open to optimizations which assume termination.

That said, an issue arises with the current wording in C, because this loop:

```
while (cond) {
  // ...
}
```

is not equivalent to that loop:

```
while (true) {
  if (!cond)
    break;
  // ...
}
```

Because its controlling expression is not a constant expression. This difference is surprising because the loops look syntactically equivalent. The same applies for any loop which has a constant expression controlling expression that converts to `true`, and has a `return`, `longjmp`, or (potentially nested) `throw`.

We could conside a change such as adding an implicit `std::this_thread::yield()` to all loops whose controlling expression is a constant expression, but doing so would needlessly pessimize code. This pessimization would likely be measured in multiples for GPU code. Indeed, without the `yield` some GPU hardware would halt forward progress on *all* warps in a thread block if one of the warps were to have an infinite loop. This behavior is acceptable while infinite loops are Undefined Behavior, but pessimizes loops which aren't infinite (and therefore don't exhibit forward progress issues).

The approach with `yield` also causes problems pertaining to code motion: a compiler is no longer allowed to hoist code from below a loops whose controlling expression is a constant expression to above it, nor merge loops whose whose controlling expression is a constant expression. This is also a pessimization which affects more than "trivial" infinite loops.

A change is therefore required which does not match the C semantics, and C would then need to be updated to match. It should be pointed out that C technically does not have forward progress guarantees, but hints at maybe having some guarantees, thefore changes to C ought to consider also adding forward progress guarantees.

The proposed wording captures very narrowly a few forms of infinite loops and makes them equivalent to a call to a new function, `std::this_thread::halt();`, which is freestanding. By doing so, the proposal does not pessimize certain types of loops as C does, and continues to allow an implementation to assume that most loops will terminate.

The proposed wording also adds to the list of allowed assumptions that a `[[noreturn]]` function or `std::this_thread::yield()` will be called. They were a missing form of valid non-forward-progress in threads that was discovered while discussing this paper in SG1.

This paper does not affect other forms of backwards control flow such as `goto`, `setjmp`/`longjmp`, infinite tail recursion, nor looping with signal handlers. While they are also forms of loops, infinite versions of them are not idiomatic in C and

C++. Functional programmers might be upset at this fact.

§ 6. Library Wording

Modify **[utility.syn]** as follows:

> The header `<utility>` contains some basic function and class templates that are used throughout the rest of the library.
>
> ```
> namespace std {
>
>   // ...
>
>
>   // [utility.unreachable], unreachable
>   [[noreturn]] void unreachable();
>
>
>   // [utility.halt], halt
>   namespace this_thread {
>     [[noreturn]] void halt() noexcept;
>   }
>
>   // ...
>
> }
> ```

Also modify **[thread.syn]** as above.

Add a new section **[utility.halt]** as follows:

> 22.2.☐ Function halt [utility.halt]
>
> ```
> namespace this_thread {
>   [[noreturn]] void halt() noexcept;
> }
> ```
>
> *Effects*: Equivalent to:
>
> ```
> while (true)
>   std::this_thread::yield();
> ```

§ 6.1. Library Bikeshed

`std::this_thread::halt` could be named something else. Here are suggestions:

- `std::infinite_loop`
- `std::block_forever`
- `std::sleep_forever`
- `std::hang`
- `std::sleep`
- `std::stop`
- `std::pause`
- `std::suspend`
- `std::freeze`
- `std::busy`
- `std::stall`
- `std::restrain`

- `std::hamster_wheel`
- `std::beach_ball`
- `std::chill_pill`
- `std::hold_your_horses`
- `std::🔁`
- `std::🎡`

§ 7. Language Wording Option #1

Modify **[intro.progress]** as follows:

> The implementation may assume that any thread will eventually do one of the following:
>
> - terminate,
> - call a function marked `[[noreturn]]`,
> - call `std::this_thread::yield()`,
> - make a call to a library I/O function,
> - perform an access through a `volatile` glvalue, or
> - perform a synchronization operation or an atomic operation.
>
> [*Note:* This is intended to allow compiler transformations such as removal , merging, and reordering of empty loops, even when termination cannot be proven. —*end note*]
>
> Notwithstanding the above, the implementation must implement an iteration statement (*[stmt.iter]*) by a call to `std::this_thread::halt()` if this iteration statement:
>
> - does not perform one of the above, and
> - the iteration statement's controlling expression is a converted constant expression (*[expr.const]*) that converts to `true`, where the controlling expression is the *condition* of a `while` or `for` loop, or the `expression` of a `do`/`while` loop, and
> - the iteration statement contains none of:
>   - a top-level `break` statement,
>   - a `goto` statement which goes out of the loop,
>   - a `return`,
>   - a `co_return` statement,
>   - a `co_await` expression,
>   - a `co_yield` expression, and
> - the iteration statement doesn't contain any of:
>   - an exception being thrown,
>   - a call to `std::longjmp()`.
>
> [*Note:* This is intended to allow programmers to use idiomatic trivial infinite loops without these loops being Undefined Behavior per the previous clause. —*end note*]

§ 8. Language Wording Option #2

Modify **[intro.progress]** as follows:

> The implementation may assume that any thread will eventually do one of the following:
>
> - terminate,
> - call a function marked `[[noreturn]]`,
> - call `std::this_thread::yield()`,
> - make a call to a library I/O function,
> - perform an access through a `volatile` glvalue, or
> - perform a synchronization operation or an atomic operation.
>
> [*Note:* This is intended to allow compiler transformations such as removal , merging, and reordering of empty loops, even when termination cannot be proven. —*end note*]
>
> Notwithstanding the above, the implementation must implement an iteration statement (*[stmt.iter]*) by a call to `std::this_thread::halt()` if this iteration statement:
>
> - does not perform one of the above, and
> - the iteration statement's controlling expression is a converted constant expression (*[expr.const]*) that converts to `true`, where the controlling expression is the *condition* of a `while` or `for` loop, or the `expression` of a `do`/`while` loop, and
> - the iteration statement's statement is empty.
>
> [*Note:* This is intended to allow programmers to use idiomatic trivial infinite loops without these loops being Undefined Behavior per the previous clause. —*end note*]

§ 9. Effect

Here are examples of the effects of this proposed change:

Code	Current equivalence in C++	Current equivalence in C	Equivalenc
`while (true)` `    /* nothing */;`	undefined behavior `std::unreachable();`	infinite loop (same as `std::this_thread::halt();`)	`std::thi`
`for (;;)` `    /* nothing */;`	undefined behavior `std::unreachable();`	infinite loop (same as `std::this_thread::halt();`)	`std::thi`
`while (true)` `    "a meaningless statement";`	undefined behavior `std::unreachable();`	infinite loop (same as `std::this_thread::halt();`)	`std::thi`

`bool done{false};`
`while (!done)`
`    /* nothing */;`
	undefined behavior
	`std::unreachable();`
		undefined behavior
		`std::unreachable();`
			undefined beha
			`std::unr`

`constexpr bool done{false};`
`while (!done)`
`    /* nothing */;`
	undefined behavior
	`std::unreachable();`
		infinite loop (same as `std::this_thread::halt();`)	infinite loop (s
			`std::this_t`

`bool done{false};`
`while (true)`
`    if (done)`
`        break;`
	undefined behavior
	`std::unreachable();`
		infinite loop (same as `std::this_thread::halt();`)	undefined beha
			`std::unr`

`while (true)`
`    if constexpr (false)`
`        break;`
	undefined behavior
	`std::unreachable();`
		infinite loop (same as `std::this_thread::halt();`)	infinite loop (s
			`std::this_t`

Code	Current equivalence in C++	Current equivalence in C	Equivalenc

`void external();`
`while (true)`
`  external();`
	unchanged (Undefined Behavior if `external` doesn't contain a side-effect)
	`void external();`
	`while (true)`
	`  external();`
		unchanged (Undefined Behavior if `external` doesn't contain a side-effect)
		`void external();`
		`while (true)`
		`  external();`
			unchanged (Un doesn't contain
			`void ext`
			`while (t`
			`  exte`

`while (true)`
`  std::this_thread::yield();`
	undefined behavior
	`std::unreachable();`
		unchanged
		`while (true)`
		`  thrd_yield();`
			unchanged
			`while (t`
			`  std:`

§ 10. Alternative

An alternative would be to specify something along the lines of:

> The implementation may assume that any thread will eventually do one of the following:
>
> - terminate,
> - call a function marked `[[noreturn]]`,
> - call `std::this_thread::yield()`,
> - make a call to a library I/O function,
> - perform an access through a `volatile` glvalue, or
> - perform a synchronization operation or an atomic operation.
>
> [*Note:* This is intended to allow compiler transformations such as removal , merging, and reordering of empty loops, even when termination cannot be proven. —*end note*]
>
> Notwithstanding the above, a program containing an iteration statement's controlling expression is a converted constant expression (*[expr.const]*) that converts to `true`, where the controlling expression is the *condition* of a `while` or `for` loop, or the `expression` of a `do`/`while`, which is dynamically executes, but which does not contain one of the above, has no forward progress guarantee until the iteration statement terminates, at which point its forward progress guarantees are restored.

This approach would cast a wider net, and allow SIMT implementation or cooperative multithreading implementation on a uniprocessor to fail to manifest forward progress on long-running loops which don't have side-effects yet do terminate.

§ References

§ Informative References

[ForwardProgress]
Olivier Giroux. Forward Progress in C++. 2022-09-12. URL: https://www.youtube.com/watch?v=CuWM-OrPitw

[LLVMProgressDiscussion]
Nikita Popov. [LangRef] Document forward-progress requirement (Abandoned). 2019-08-04. URL: https://reviews.llvm.org/D65718

[N1349]

Clark Nelson; Hans-J. Boehm; Lawrence Crowl. Parallel memory sequencing model proposal. 2009-02-23. URL: https://www.open-std.org/jtc1/sc22/wg14/www/docs/n1349.htm

[N1509]

Douglas Walls. Optimizing away infinite loops. 2010-09-02. URL: https://www.open-std.org/jtc1/sc22/wg14/www/docs/n1509.pdf

[N1528]

Hans Boehm. Why undefined behavior for infinite loops?. 2010-10-27. URL: https://www.open-std.org/jtc1/sc22/wg14/www/docs/n1528.htm

[N2644]

Jens Gustedt. Programming languages — a common C/C++ core specification. 2021-01-20. URL: https://www.open-std.org/jtc1/sc22/wg14/www/docs/n2644.pdf

[P2735r0]

Aaron Ballman. C xor C++ Programming. 5 December 2022. URL: https://wg21.link/p2735r0

[P2809r0]

JF Bastien. Trivial infinite loops are not Undefined Behavior. 15 March 2023. URL: https://wg21.link/p2809r0